

Agents for Solving the Battleship Puzzle

A simple reflex agent, a goal-based agent and an agent based on achieving optimal performance

Luis Alberto Jiménez Román Ramírez Martín Edgar Alan González Falcón Luis Adrián
López Morales Fernando Samuel

September 27, 2026

Universidad Nacional Autónoma de México – Facultad de Ingeniería
Artificial Intelligence (0406), Group 2 – Prof. José Jaime Camacho Escoto

Play with our agents right now

[figure to add: figuras/fig_qr.png]

Figure 1: Scan the code and play against the agents on your phone.

https://theuniquefersa.github.io/AI-Lab_1-Agents/

Goals

- Formulate and create three different kinds of agents:
 - **SRA**: Simple Reflex Agent
 - **GBA**: Goal-Based Agent
 - **ABAOP**: Agent Based on Achieving Optimal Performance
- Test each agent against agents of the same and of a different kind.
- Build an application that anybody can use, also from a phone, where the choices of the agents are seen graphically and the time (or each single step) can be controlled.
- Obtain statistics from a large number of games, so the conclusions are based on data.
- Validate the simulation against theory and study the computational cost of each agent.

The problem: search under partial observability

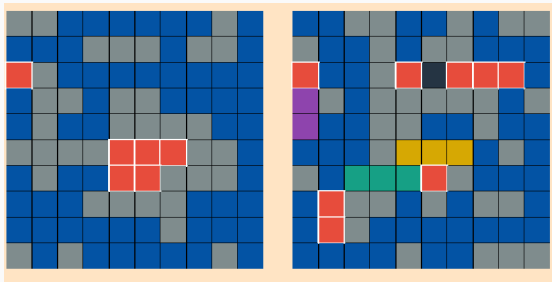


Figure 2: 10×10 board: 17 ship cells hidden among 100.

PEAS (Russell and Norvig):

- **P**: shots needed to sink the fleet (fewer is better).
- **E**: partially observable, discrete, static, sequential.
- **A**: fire at one cell.
- **S**: hit / miss, and the notice that a ship sank.

Each agent shoots at **its own board**: a game is a **race** between two independent runs.

Three kinds of agents (Russell and Norvig)

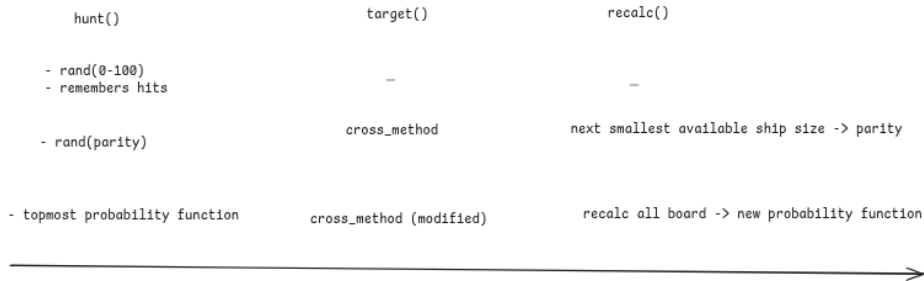


Figure 3: Simple reflex, goal based and utility based, mapped to our three agents.

SRA → simple reflex
condition-action rule

GBA → goal based
goal: sink the ship found

ABAOP → utility based
maximise a performance measure

SRA: shoot at random (our baseline)

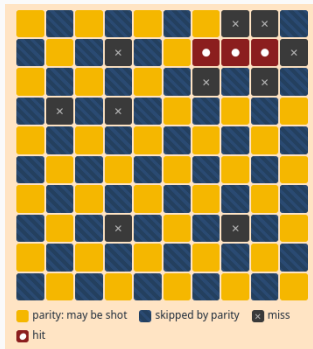


Figure 4: Every cell not shot yet has probability $1/|U|$.

It ignores what it learns, so its behaviour has a **closed form**: with $N = 100$ cells and $K = 17$ ship cells,

$$P(X = m) = \frac{\binom{m-1}{K-1}}{\binom{N}{K}}$$

$$E[X] = \frac{K(N+1)}{K+1} = 95.39$$

- It needs almost the whole board.
- It needs all 100 shots in 17 % of the games (also reported by Berry).
- We use it to **validate the simulator**.

GBA: parity search + directed sinking

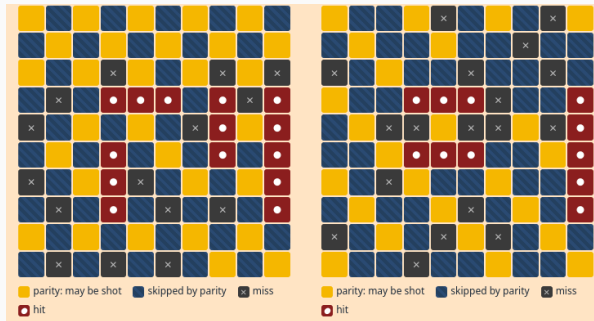


Figure 5: Parity $p = 2$ (50 cells) and $p = 3$ (34 cells).

Hunt. Only cells with $(r - c) \bmod p = 0$, where p is the shortest ship afloat: no ship can hide from that set (Berry).

Target. From the hit it explores Up–Right–Down–Left, locks the axis at the first new hit, advances until a miss, then returns to the origin.

Our variation follows the line of the ship instead of walking around every hit.

ABAOP: fire at the most probable cell



Figure 6: Density map: how many legal placements cover each cell.

For every ship afloat and every legal placement π ,

$$w(\pi) = \begin{cases} 1 & \text{hunt} \\ 10^{h(\pi)} & \text{target} \end{cases}$$

$$D(c) = \sum_{c \in \pi \text{ legal}} w(\pi)$$

and it shoots at $\arg \max D(c)$ (Berry; popularised by Vsauce2).

No hand-made rules: the parity, the neighbours and the change of direction emerge from one formula.

The application: watch the agent think

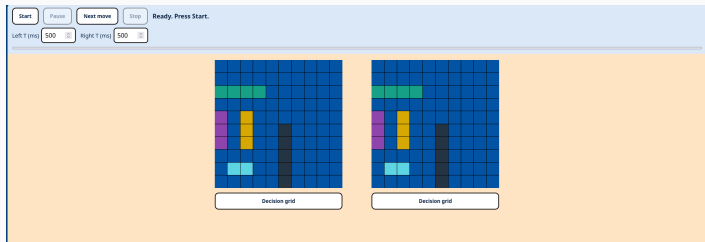


Figure 7: Boards and *decision grids*: parity (GBA) and density (ABAOP).

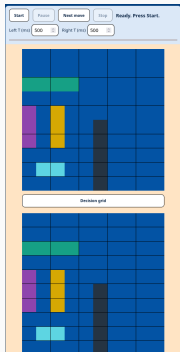


Figure 8: Responsive: it also runs on a phone.

JavaScript + HTML + CSS on GitHub Pages · *Next move* step by step · delay T live · $T = 0$:
10 000 games and a CSV.

How we made the comparison fair

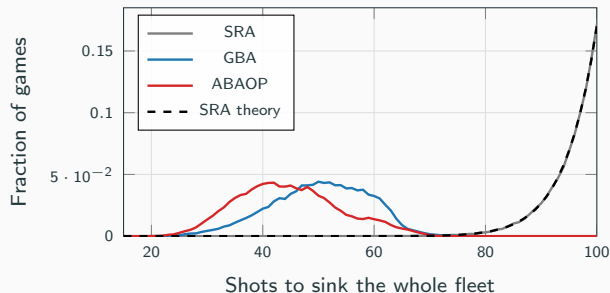
**7 contests × 10 000 games =
70 000 games**

1. **Same fleet layout** for both agents: the difference is the agent, not the luck of the layout.
2. **The first shooter alternates**: on a tie the first to shoot wins, and that bias has to cancel.
3. **The loser plays out**: otherwise its shots are cut short and its average is biased.

#	Contest	Layout
1	SRA vs SRA	same
2	SRA vs SRA	<i>different</i>
3	GBA vs GBA	same
4	ABAOP vs ABAOP	same
5	SRA vs GBA	same
6	SRA vs ABAOP	same
7	GBA vs ABAOP	same

Data integrity: 100 % of the 70 000 records
consistent.

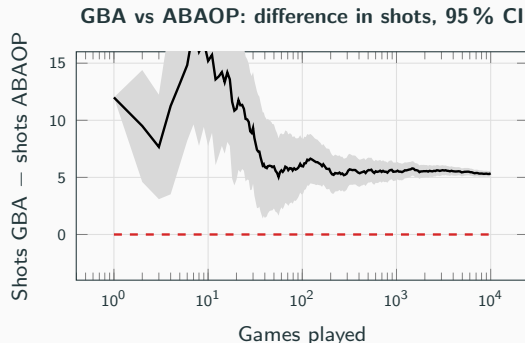
How many shots does each agent need?



Agent	Mean $\pm$ sd	Median
SRA	95.4 $\pm$ 4.8	97
GBA	50.3 $\pm$ 8.7	51
ABAOP	44.7 $\pm$ 9.2	44

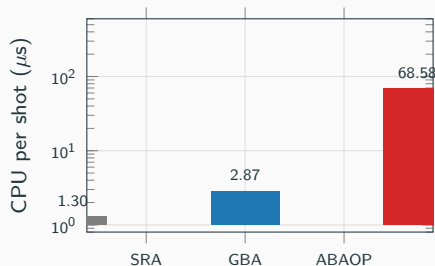
- GBA saves 47 % of the shots of the SRA; ABAOP 53 %.
- The SRA matches its theory (χ^2 , $p = 0.68$): the simulator is correct.

Who wins? The better agent still loses 1 game in 3



- ABAOP vs GBA: 67.0 % of wins (CI 66.1–67.9), 5.36 ± 0.24 shots fewer per game.
- SRA won 1 of 20 000 games against them.
- Same kind vs same kind: $\approx 50\%$ each, as expected.
- The advantage is real but small against the noise of one game: 5 shots, sd of 12.5.

What does the intelligence cost?



Agent	Per shot	Per game	Complexity
SRA	1.30 μs	0.12 ms	$O(1)$
GBA	2.87 μs	0.14 ms	$O(1) + \text{recalc}$
ABAOP	68.58 μs	3.07 ms	$\Theta(NS)$

- ABAOP rebuilds the whole map every shot:
 ≈ 1449 cell checks (N cells $\times$ fleet length S).
- 53 $\times$ the cost of the SRA ... but still only 3 ms per game.

Our numbers against the authors

Median shots	Berry (DataGenetics, 2011)	Schwartz (2022)	Our result
Random (SRA)	97	97	97
Hunt/target + parity (GBA)	64	—	51
Probability density (ABAOP)	42	42	44

Where we agree. The random agent reproduces the published value exactly, and our density agent lands within two shots of Berry's.

Where we differ. Our GBA beats the published hunt/target by **13 shots**, because it follows the *line* of the ship instead of walking around every hit.

Conclusions

- **ABAOP is the best agent:** 44.7 shots, 67 % of wins against the GBA. *But* it wins two games out of three, not all of them: in a single game luck is still important.
- **GBA is the best balance:** it gets 88 % of the benefit for 4 % of the cost. For a bigger board, a phone, or thousands of decisions per second, it is the reasonable choice.
- **Is the ABAOP worth it?** Yes here: 3 ms per game is nothing. The step SRA→GBA saves 45 shots almost for free; the step GBA→ABAOP saves 5.6 more and costs a thousand times more per shot saved.
- **The design of the experiment matters as much as the agent:** the same layout, the alternating first shooter and letting the loser finish were needed to avoid biased numbers.
- **Validation:** the SRA agrees with its closed form ($p = 0.68$), which is what lets us trust the rest of the numbers.

References

-  N. Berry, “Battleship,” *DataGenetics*, Dec. 2011. [Online]. Available: datagenetics.com/blog/december32011/
-  A. Schwartz, “Coding an intelligent Battleship agent,” *Towards Data Science*, 2022.
-  Vsauce2, “The Battleship algorithm,” YouTube, 2019.
-  S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
-  T. H. Cormen *et al.*, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

Questions?

https://theuniquefersa.github.io/AI-Lab_1-Agents/

Backup: advantage of the first shooter

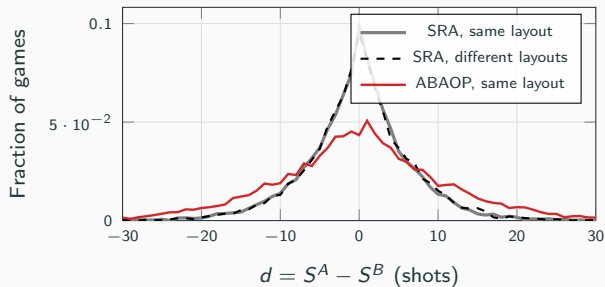
On a tie the agent that shot first wins, so

$$P(\text{first shooter wins}) = \frac{1 + P(\text{tie})}{2}$$

Contest	$P(\text{tie})$	Predicted	Observed
SRA vs SRA	9.84 %	54.92 %	53.80 %
GBA vs GBA	3.29 %	51.64 %	51.25 %
ABAOP vs ABAOP	4.34 %	52.17 %	52.87 %
SRA vs ABAOP	0.00 %	50.00 %	50.00 %

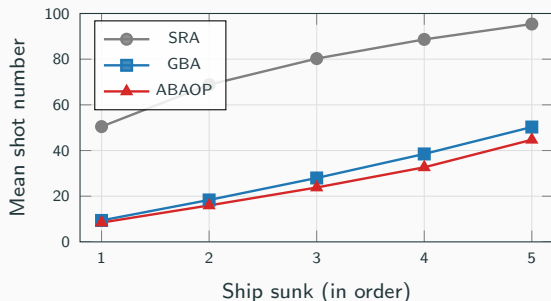
That is why the first shooter has to alternate: otherwise one side starts with up to 5 points of advantage.

Backup: does the fleet layout matter?



- For the **SRA** the layout changes nothing ($\rho = 0.03$): the two curves overlap.
- For the **ABAOP** the layout explains 29 % of the variance ($\rho = 0.29$): some layouts really are harder for it.
- Using the same layout reduces the variance of the comparison by a factor of 1.41.

Backup: when does each ship sink?



- The ABAOP builds its advantage between the **second and the fourth ship**.
- For the **last ship** both agents need the same: the remaining empty area has to be searched anyway.
- Hit rate: SRA 17.9 %, GBA 35.0 %, ABAOP 39.7 %.